A brief review of software verification for quantum programs

Genghan Zhang

zhang-gh19@mails.tsinghua.edu.cn

June 9, 2022

Abstract

Quantum program software verification is an important but challenging task. This article briefly reviews formal proofs methods and simulation methods for quantum program software verification. Formal proofs methods mainly have three categories and simulation methods have two main research directions.

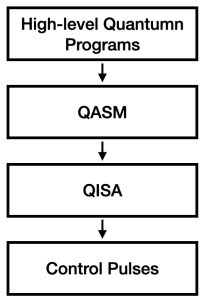
1 Introduction

As new quantum algorithms and new quantum computing devices emerge [1–4], more reliable and scaleble quantum computation systems are needed. Besides the theoretical challenge which requires a quantum algorithm to be at least polynomial faster than the classical counterpart [5], quantum algorithm designers also have to face the fact that there's no panacea for developing and verifying their algorithms, not only because different quantum computing devices have different controlling methods, but also the real devices that can run the algorithm may haven't emerged [6, 7].

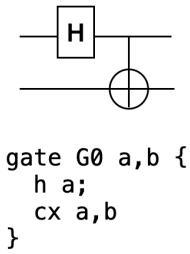
Quantum programming languages(QPLs)[1] are designed to tackle these challenges. In a broad sense, QPLs express quantum algorithms and how they are lowered to instructions that can be exactly executed on a real quantum computing device [9]. Currently, QPLs mainly have three levels, as shown in figure 1a. Quantum Programs can describe quantum algorithms [10–12], some programs can also describe the control of the host CPU [13] or the device constraints [14]. QASM is a gate-level desciption language that specifies 1- and 2-qubit operations [15, 16]. Then the QASM will be translated to QISA which basically contains instructions that control the pulses executated on quantum computing devices.

Writing a correct quantum program is an essential but challenging task. More and more quantum programming languages are being developed to deal with this challenge. A key question is: how to verify the program? This task is termed *software verification*. Software verification has two main purposes: (i) high-level quantum programs are correct syntactically and semantically [17–22] (ii) transformation on QPLs preserve equivalence [23–25]. Mainly three methods are used to achieve these two goals: (i) proofs via formal verification (ii) tracing via classical simulation (iii) quantum property testing. We will review the first two methods, and invite readers interested in the third method to refer to[26], because the first two methods are more closely related with the software verification.

[1] Use QPLs to distinguish from QPL in [8]



(a) QPLs stack [27]



(b) QASM example [15]

Figure 1: QPLs explanation

2 Formal proofs methods

Formal proofs express a quantum program in a well-defined semantics and use classical formal verification tools to efficiently verify the program. It's an open problem how to formalize the quantum circuit characteristics, such as the fragile qubits, no-cloning property, measurement, equivalent transformation of the gates (an example in figure2) into classical logic [5]. Many tools have been developed to do formal proofs on quantum circuits. I will categorize them based on the classical logic framework they are reduced to, because such final logic will express how the essential quantum concepts in circuits are modeled [28]. This section is organized as follow: each category's motivation is introduced first and then the core idea is explained. In order to emphasize the insights behind them, detailed explanation of syntax and semantics is omitted. To the best of the author's knowledge, no article has classified formal proofs methods from this view.

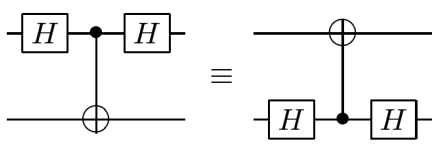


Figure 2: Equivalent transformation example

2.1 SMT

2.1.1 Motivation

Satisfiability Modulo Theories (SMT) refers to the problem of determining whether a first-order formula is satisfiable with respect to some logical theory[29]. For example, an SMT solver may want to check if

$$(x + y < 0) \wedge (x < 0) \wedge (y < 0) \tag{1}$$

is satisfiable. Different from Boolean satisfiability problem (SAT), SMT requires *background theory*. Different from general-purpose reasoning methods that want to find whether there exists a nonstandard interpretation of the symbols $+, <, 0$ that makes 1 satisfiable, practical SMT solvers just need to verify 1 under the assumption that $<$ is the usual ordering over the real number, $+$ the usual addition and 0 the identity of addition.

Such nature of SMT fits the QPLs verification well in the models of [23, 24]. In general, they use denotational semantics to model the quantum circuit and formulate the equivalent transformations as the background theory, then a circuit and the compiler passes exerted on them can be written to a first-order formula. Such formula can be solved efficiently using state-of-art SMT solvers, such as Z3[30].

2.1.2 Core idea

[23, 24] both intend to detect bugs of compiler passes in [31]. Such task has three main challenges. (i) Quantum compilers have to obey quantum mechanics rules which are complicated and counter-intuitive (ii) Quantum compilers rely heavily on the optimization passes to fulfill the device and noise budgets[32] (iii) Verification methods have to be easily scaled and automated, because the compiler code base keeps growing[31].

For example, in CertiQ[23] a verification can be modeled as below. For a compiler pass $app(C; CX; CX; T(C^{'}), Q)$. The claim G is $app(C; C^{'}, Q) \equiv app(C; CX; CX; T(C^{'}), Q)$. Two adjacent CX gates will cancel, so the first background theory P_1 is $\forall Q_1, app(CX; CX, Q) \equiv Q_1$. T is an equivalent transformation from the CertiQ library, so the second background theory P_2 is $\forall C_2\ Q_2, app(T(C_2), Q_2) \equiv app(C_2, Q_2)$. CertiQ can encode them into an SMT formula and feed it to Z3 to check $P_1 \wedge P_2 \wedge \neg Q$. CertiQ also designs an equivalence checker between different data representations for qubits, and a whole code stack to support both verification and execution at as low user burden as possible. Because SMT doesn't support unbounded loop structure, CertiQ also distills three loop patterns from most of Qiskit compiler passes. Giallar[24] builds upon CertiQ to enable a push-button style verification of the compiler passes.

2.2 Linear Type Systems

2.2.1 Motivation

Linear logic[33] restricts times premises are used to derive conclusions. To be specific, every premise has to be used exactly **once** to construct the conclusion. For example, if we have three premises $A \rightarrow B, A \rightarrow C$, and A in ordinary logic, it's safe to derive $A \mapsto B \wedge C$, even though A is used twice to derive the final logic. However, in linear logic the final logic is wrong because A is used twice. Instead, it allows $A, A \mapsto B \bigotimes C$ and $B \bigotimes C$ means it has both B and C. This article[2] uses a "chemical reaction matter-conserving" metaphor to describe linear logic. Such logic, in its nature, extremely resembles the quantum circuit where a qubit can only be used **once** if we don't consider the compilcated and error-prone qubit resue process.

2.2.2 Core idea

[20, 21] both use linear type system to assist verification proofs. In Qwire[21] wires are treated linearly. That means a wire can be only used once. The following Qwire code describes the qubit cloning and is wrong because its equivalent logic $W \mapsto W, W$ violates the "matter-conserving" nature.

```
Definition clone W : Box W (W * W).
    box_ w => output (w,w)
```

Compared with CertiQ which only uses Coq[34] to prove the rewriting roles, Qwire is implemented natively in Coq. Each wire is described by a wire type and gates are indexed by a pair

[2] https://www.cs.cornell.edu/courses/cs6110/2017sp/lectures/lec30.pdf

of wire types. A wire type is constructed with patterns which consist of basic WTypes. Coq's dependent types can be used to constrain how patterns can be constructed and thus only allow legal wire types. To verify general quantum programs VOQC[20] designs SQIR and extends the Qwire. The key difference between Qwire and SQIR is how they define qubits: SQIR uses concrete indices into a global register while Qwire uses abstract type variables. Concrete indices make it easier to reason about qubits because they are naturally mapped to columns and rows in a density matrix. However, they bring about much inconvenience when compositing closed circuits. Quipper[13], Proto-Quipper-S[35], Proto-Quipper-M[17] and Proto-Quipper-D[36] are also based on a linear type system with extra QPLs related concepts that help verification. For example, Proto-Quipper-M starts by defining parameters and state, and defines categorical structures in the model. Such design philosophy ensures the language is "correct by design".

2.3 Floyd-Hoare logic

2.3.1 Motivation

The goal of Hoare logic is to provide a compositional method for proving the validity of specific Hoare triples. The standard notation of a Hoare triple is $\{P\}c\{Q\}$, which means if command c begins execution in a state satisfying assertion P, and if c eventually terminates in some final state, then that final state will satisfy the assertion Q[37]. P is called precondition and Q is called postcondition. Quantum programs can fit into such general logic framework. Basically, [18] defines the correctness formula $\models_{par} \{P\}S\{Q\}, \models_{tot} \{P\}S\{Q\}$ of a quantum program S as the inequality between P and Q. The precondition and the postcondition are defined as $tr(P\rho)$ and $trQ[S](\rho)$.

2.3.2 Core idea

[18, 19, 25] all use the Hoare logic framework to describe the verification problem. QHL [18] models quantum programs as super-operators and a quantum predicate as an Hermitian operator. In other words, P and Q in the Hoare triple are two quantum predicates and c is a quantum program. Based on the denotational semantics of quantum programs, the notions of partial and total correctness can be defined, which can further defines the weakest and the weakest liberal preconditions. aQHL [19] develops a practical variant of QHL. It restricts P and Q to projection operators which simplifies the inference rules. Besides, it develops several rules of reasoning about robustness of quantum programs to prove approximate satisfaction of a postcondition.

3 Simulation Methods

Such methods do simulation of quantum programs on a classical computer. Based on what the simulation returns, they can be divided into two categories: strong and weak [26]. Strong simulation returns the probabilities of the output measurement outcomes, while weak simulation only returns a sample of the measurement outcomes.

Designing a simulation algorithm is challenging, because it must simulate a quantum program both with high accuracy and with low error rate. What's more, for fairly large-scale quantum programs, it is infeasible to directly record state vectors at every simulation time step because of the exponential memory consumption. Therefore, most researches choose to simulate programs with specific structures and figure out simulation algorithms that fit the structures. For example, theoretically, for stabilizer circuits (only consisted of gates generated from Clifford group), there exists efficient classical simulation [38, 39].

3.1 Classical big data techniques help

Naive density matrix simulation algorithm requires exponential time and space cost. For a depth n circuit with m-qubit(no entanglement), a unitary gate is a $2^m \times 2^m$ matrix. So if we use naive matrix multiplication to simulate the unitary gate, we need to do $O(n2^{2m})$ arithmetic operations and store a $O(m)$ state vector. However, some big data algorithm from classical computation, such as data compression and distributed computing can be applied to accelerate the simulation [40]. Besides, researchers has surprisingly some circuits where all qubits are entangled, but still can be simulated efficiently on a classical computer [41].

3.2 New notions open new opportunities

Tensor network method use a tensor network to represent a quantum circuit, and gates are modeled as tensor network contraction [26]. Therefore, the challenge is transformed to efficient tensor contraction algorithm which is a well researched area [42]. Combined with Feynman's sum-over-path approach, undirected graphical model is established to further optimize the tensor contraction algorithm [43]. Other new models, such as ZX-diagrams [44, 45] have also been developed to accelerate simulation.

4 Other works

Because this review aims to summarize some insights when designing verification methods for QPLs, formal form of theoretical foundations of the mentioned methods aren't included. For example, two fundamental theorems: Exact compilation theorem and Solovay-Kitaev theorem [46] that make quantum compilation possible aren't discussed in this review. Besides, improvements on methods in this review are been developed. For example, Proq [47] uses projection operators to design runtime assertions. [48] further extends Proto-Quipper-M to use the linear dependent type theory. What's more, researchers are developing new IRs for QPLs to optimize quantum program correctly[49, 50]. Interestingly, the name "Paulihedral" is composed of "Pauli" and "Polyhedral". And polyhedral model[51] is a type of classical compiler optimization. Quartz[50] also uses TASO[52], an optimization tool for computation graph in classical programs to optimize quantum programs. It's reasonable to expect more knowledge from classical computer science to improve QPLs.

5 Conclusion

This article briefly reviews two software verification methods: formal proofs methods and simulation methods. Formal proofs methods can be categorized into three types according to the classical logic framework they are reduced to: SMT, linear type systems and Floyd-Hoare logic. All have their own motivation to be used. Simulation methods have two takeaway messages: classical big data techniques help and new notions open new opportunities. Due to the restriction of author's knowledge and the space, QPLs for each verification method are not fully listed and advanced discussions on the verification theory are not included.

References

[1] Han-Sen Zhong, Hui Wang, Yu-Hao Deng, Ming-Cheng Chen, Li-Chao Peng, Yi-Han Luo, Jian Qin, Dian Wu, Xing Ding, Yi Hu, et al. Quantum computational advantage using photons. *Science*, 370(6523):1460–1463, 2020.

[2] Michel H Devoret and Robert J Schoelkopf. Superconducting circuits for quantum information: an outlook. *Science*, 339(6124):1169–1174, 2013.

[3] Bjoern Lekitsch, Sebastian Weidt, Austin G Fowler, Klaus Mølmer, Simon J Devitt, Christof Wunderlich, and Winfried K Hensinger. Blueprint for a microwave trapped ion quantum computer. *Science Advances*, 3(2):e1601540, 2017.

[4] Daniel Loss and David P DiVincenzo. Quantum computation with quantum dots. *Physical Review A*, 57(1):120, 1998.

[5] Michael A Nielsen and Isaac Chuang. Quantum computation and quantum information, 2002.

[6] A Yu Kitaev. Quantum measurements and the abelian stabilizer problem. *arXiv preprint quant-ph/9511026*, 1995.

[7] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Contemporary Mathematics*, 305:53–74, 2002.

[8] PETER SELINGER. Towards a quantum programming language. *Mathematical Structures in Computer Science*, 14(4):527–586, 2004. doi: 10.1017/S0960129504004256.

[9] Bettina Heim, Mathias Soeken, Sarah Marshall, Chris Granade, Martin Roetteler, Alan Geller, Matthias Troyer, and Krysta Svore. Quantum programming languages. *Nature Reviews Physics*, 2(12):709–722, 2020.

[10] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. t— ket¿: a retargetable compiler for nisq devices. *Quantum Science and Technology*, 6(1):014003, 2020.

[11] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. Q# enabling scalable quantum computing and development with a high-level dsl. In *Proceedings of the real world domain specific languages workshop 2018*, pages 1–10, 2018.

[12] Ali J Abhari, Arvin Faruque, Mohammad J Dousti, Lukas Svec, Oana Catu, Amlan Chakrabati, Chen-Fu Chiang, Seth Vanderwilt, John Black, and Fred Chong. Scaffold: Quantum programming language. Technical report, Princeton Univ NJ Dept of Computer Science, 2012.

[13] Alexander S Green, Peter LeFanu Lumsdaine, Neil J Ross, Peter Selinger, and Benoît Valiron. Quipper: a scalable quantum programming language. In *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*, pages 333–342, 2013.

[14] Cirq Developers. Cirq, August 2021. URL https://doi.org/10.5281/zenodo.5182845. See full list of authors on Github: https://github .com/quantumlib/Cirq/graphs/contributors.

[15] Andrew W Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S Bishop, Steven Heidel, Colm A Ryan, John Smolin, Jay M Gambetta, and Blake R Johnson. Openqasm 3: A broader and deeper quantum assembly language. *arXiv preprint arXiv:2104.14722*, 2021.

[16] Robert S Smith, Michael J Curtis, and William J Zeng. A practical quantum instruction set architecture. *arXiv preprint arXiv:1608.03355*, 2016.

[17] Francisco Rios and Peter Selinger. A categorical model for a quantum circuit description language. *arXiv preprint arXiv:1706.02630*, 2017.

[18] Mingsheng Ying. Floyd–hoare logic for quantum programs. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 33(6):1–49, 2012.

[19] Li Zhou, Nengkun Yu, and Mingsheng Ying. An applied quantum hoare logic. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 1149–1162, 2019.

[20] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. A verified optimizer for quantum circuits. *arXiv preprint arXiv:1912.02250*, 2019.

[21] Robert Rand, Jennifer Paykin, and Steve Zdancewic. Qwire practice: Formal verification of quantum circuits in coq. *arXiv preprint arXiv:1803.00699*, 2018.

[22] Runzhou Tao, Yunong Shi, Jianan Yao, John Hui, Frederic T Chong, and Ronghui Gu. Gleipnir: toward practical error analysis for quantum programs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 48–64, 2021.

[23] Yunong Shi, Runzhou Tao, Xupeng Li, Ali Javadi-Abhari, Andrew W Cross, Frederic T Chong, and Ronghui Gu. Certiq: A mostly-automated verification of a realistic quantum compiler. *arXiv preprint arXiv:1908.08963*, 2019.

[24] Runzhou Tao, Yunong Shi, Jianan Yao, Xupeng Li, Ali Javadi-Abhari, Andrew W Cross, Frederic T Chong, and Ronghui Gu. Giallar: Push-button verification for the qiskit quantum compiler. *arXiv preprint arXiv:2205.00661*, 2022.

[25] Dominique Unruh. Quantum relational hoare logic. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–31, 2019.

[26] Yongshan Ding and Frederic T Chong. Quantum computer systems: Research for noisy intermediate-scale quantum computers. *Synthesis Lectures on Computer Architecture*, 15(2): 1–227, 2020.

[27] Yunong Shi, Nelson Leung, Pranav Gokhale, Zane Rossi, David I Schuster, Henry Hoffmann, and Frederic T Chong. Optimized compilation of aggregated instructions for realistic quantum computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 1031–1044, 2019.

[28] Kenneth E Iverson. Notation as a tool of thought. In *ACM Turing award lectures*, page 1979. 2007.

[29] Clark Barrett and Cesare Tinelli. Satisfiability modulo theories. In *Handbook of model checking*, pages 305–343. Springer, 2018.

[30] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.

[31] MD SAJID ANIS, Abby-Mitchell, Héctor Abraham, AduOffei, Rochisha Agarwal, Gabriele Agliardi, Merav Aharoni, Ismail Yunus Akhalwaya, Gadi Aleksandrowicz, Thomas Alexander, Matthew Amy, Sashwat Anagolum, Anthony-Gandon, Eli Arbel, Abraham Asfaw, Anish Athalye, Artur Avkhadiev, Carlos Azaustre, PRATHAMESH BHOLE, Abhik Banerjee, Santanu Banerjee, Will Bang, Aman Bansal, Panagiotis Barkoutsos, Ashish Barnawal, George Barron, George S. Barron, Luciano Bello, Yael Ben-Haim, M. Chandler Bennett, Daniel Bevenius, Dhruv Bhatnagar, Arjun Bhobe, Paolo Bianchini, Lev S. Bishop, Carsten Blank, Sorin Bolos, Soham Bopardikar, Samuel Bosch, Sebastian Brandhofer, Brandon, Sergey Bravyi, Nick Bronn, Bryce-Fuller, David Bucher, Artemiy Burov, Fran Cabrera, Padraic Calpin, Lauren Capelluto, Jorge Carballo, Ginés Carrascal, Adam Carriker, Ivan Carvalho, Adrian Chen, Chun-Fu Chen, Edward Chen, Jielun (Chris) Chen, Richard Chen, Franck Chevallier, Kartik Chinda, Rathish Cholarajan, Jerry M. Chow, Spencer Churchill, CisterMoke, Christian Claus, Christian Clauss, Caleb Clothier, Romilly Cocking, Ryan Cocuzzo, Jordan Connor, Filipe Correa, Zachary Crockett, Abigail J. Cross, Andrew W. Cross, Simon Cross, Juan Cruz-Benito, Chris Culver, Antonio D. Córcoles-Gonzales, Navaneeth D, Sean Dague, Tareq El Dandachi, Animesh N Dangwal, Jonathan Daniel, Marcus Daniels, Matthieu Dartiailh, Abdón Rodríguez Davila, Faisal Debouni, Anton Dekusar, Amol Deshmukh, Mohit Deshpande, Delton Ding, Jun Doi, Eli M. Dow, Eric Drechsler, Eugene Dumitrescu, Karel Dumon, Ivan Duran, Kareem EL-Safty, Eric Eastman, Grant Eberle, Amir Ebrahimi, Pieter Eendebak, Daniel Egger, ElePT, Emilio, Alberto Espiricueta, Mark Everitt, Davide Facoetti, Farida, Paco Martín Fernández, Samuele Ferracin, Davide Ferrari, Axel Hernández Ferrera, Romain Fouilland, Albert Frisch, Andreas Fuhrer, Bryce Fuller, MELVIN GEORGE, Julien Gacon, Borja Godoy Gago, Claudio Gambella, Jay M. Gambetta, Adhisha Gammanpila, Luis Garcia, Tanya Garg, Shelly Garion, James R. Garrison, Jim Garrison, Tim Gates, Leron Gil, Austin Gilliam, Aditya Giridharan, Juan Gomez-Mosquera, Gonzalo, Salvador de la Puente González, Jesse Gorzinski, Ian Gould, Donny Greenberg, Dmitry Grinko, Wen Guan, Dani Guijo, John A. Gunnels, Harshit Gupta, Naman Gupta, Jakob M. Günther, Mikael Haglund, Isabel Haide, Ikko Hamamura, Omar Costa Hamido, Frank Harkins, Kevin Hartman, Areeq Hasan, Vojtech Havlicek, Joe Hellmers, Łukasz Herok, Stefan Hillmich, Hiroshi Horii, Connor Howington, Shaohan Hu, Wei Hu, Junye Huang, Rolf Huisman, Haruki Imai, Takashi Imamichi, Kazuaki Ishizaki, Ishwor, Raban Iten, Toshinari Itoko, Alexander Ivrii, Ali Javadi, Ali Javadi-Abhari, Wahaj Javed, Qian Jianhua, Madhav Jivrajani, Kiran Johns, Scott Johnstun, Jonathan-Shoemaker, JosDenmark, JoshDumo, John Judge, Tal Kachmann, Akshay Kale, Naoki Kanazawa, Jessica Kane, Kang-Bae, Annanay Kapila, Anton Karazeev, Paul Kassebaum, Tobias Kehrer, Josh Kelso, Scott Kelso, Vismai Khanderao, Spencer King, Yuri Kobayashi, Kovi11Day, Arseny Kovyrshin, Rajiv Krishnakumar, Vivek Krishnan, Kevin Krsulich, Prasad Kumkar, Gawel Kus, Ryan LaRose, Enrique Lacal, Raphaël Lambert, Haggai Landa, John Lapeyre, Joe Latone, Scott Lawrence, Christina Lee, Gushu Li, Jake Lishman, Dennis Liu, Peng Liu, Lolcroc, Abhishek K M, Liam Madden, Yunho Maeng, Saurav Maheshkar, Kahan Majmudar, Aleksei Malyshev, Mohamed El Mandouh, Joshua Manela, Manjula, Jakub Marecek, Manoel Marques, Kunal Marwaha, Dmitri Maslov, Paweł Maszota, Dolph Mathews, Atsushi Matsuo, Farai Mazhandu, Doug McClure, Maureen McElaney, Cameron McGarry, David McKay, Dan McPherson, Srujan Meesala, Dekel Meirom, Corey Mendell, Thomas Metcalfe, Martin Mevissen, Andrew Meyer, Antonio Mezzacapo, Rohit Midha, Daniel Miller, Zlatko Minev, Abby Mitchell, Niko-

laj Moll, Alejandro Montanez, Gabriel Monteiro, Michael Duane Mooring, Renier Morales, Niall Moran, David Morcuende, Seif Mostafa, Mario Motta, Romain Moyard, Prakash Murali, Jan Müggenburg, Tristan NEMOZ, David Nadlinger, Ken Nakanishi, Giacomo Nannicini, Paul Nation, Edwin Navarro, Yehuda Naveh, Scott Wyman Neagle, Patrick Neuweiler, Aziz Ngoueya, Thien Nguyen, Johan Nicander, Nick-Singstock, Pradeep Niroula, Hassi Norlen, NuoWenLei, Lee James O'Riordan, Oluwatobi Ogunbayo, Pauline Ollitrault, Tamiya Onodera, Raul Otaolea, Steven Oud, Dan Padilha, Hanhee Paik, Soham Pal, Yuchen Pang, Ashish Panigrahi, Vincent R. Pascuzzi, Simone Perriello, Eric Peterson, Anna Phan, Kuba Pilch, Francesco Piro, Marco Pistoia, Christophe Piveteau, Julia Plewa, Pierre Pocreau, Alejandro Pozas-Kerstjens, Rafał Pracht, Milos Prokop, Viktor Prutyanov, Sumit Puri, Daniel Puzzuoli, Jesús Pérez, Quant02, Quintiii, Rafey Iqbal Rahman, Arun Raja, Roshan Rajeev, Isha Rajput, Nipun Ramagiri, Anirudh Rao, Rudy Raymond, Oliver Reardon-Smith, Rafael Martín-Cuevas Redondo, Max Reuter, Julia Rice, Matt Riedemann, Rietesh, Drew Risinger, Marcello La Rocca, Diego M. Rodríguez, RohithKarur, Ben Rosand, Max Rossmannek, Mingi Ryu, Tharrmashastha SAPV, Nahum Rosa Cruz Sa, Arijit Saha, Abdullah Ash-Saki, Sankalp Sanand, Martin Sandberg, Hirmay Sandesara, Ritvik Sapra, Hayk Sargsyan, Aniruddha Sarkar, Ninad Sathaye, Bruno Schmitt, Chris Schnabel, Zachary Schoenfeld, Travis L. Scholten, Eddie Schoute, Mark Schulterbrandt, Joachim Schwarm, James Seaward, Sergi, Ismael Faro Sertage, Kanav Setia, Freya Shah, Nathan Shammah, Rohan Sharma, Yunong Shi, Jonathan Shoemaker, Adenilton Silva, Andrea Simonetto, Deeksha Singh, Divyanshu Singh, Parmeet Singh, Phattharaporn Singkanipa, Yukio Siraichi, Siri, Jesús Sistos, Iskandar Sitdikov, Seyon Sivarajah, Magnus Berg Sletfjerding, John A. Smolin, Mathias Soeken, Igor Olegovich Sokolov, Igor Sokolov, Vicente P. Soloviev, SooluThomas, Starfish, Dominik Steenken, Matt Stypulkoski, Adrien Suau, Shaojun Sun, Kevin J. Sung, Makoto Suwama, Oskar Słowik, Hitomi Takahashi, Tanvesh Takawale, Ivano Tavernelli, Charles Taylor, Pete Taylour, Soolu Thomas, Kevin Tian, Mathieu Tillet, Maddy Tod, Miroslav Tomasik, Caroline Tornow, Enrique de la Torre, Juan Luis Sánchez Toural, Kenso Trabing, Matthew Treinish, Dimitar Trenev, TrishaPe, Felix Truger, Georgios Tsilimigkounakis, Davindra Tulsi, Wes Turner, Yotam Vaknin, Carmen Recio Valcarce, Francois Varchon, Adish Vartak, Almudena Carrera Vazquez, Prajjwal Vijaywargiya, Victor Villar, Bhargav Vishnu, Desiree Vogt-Lee, Christophe Vuillot, James Weaver, Johannes Weidenfeller, Rafal Wieczorek, Jonathan A. Wildstrom, Jessica Wilson, Erick Winston, WinterSoldier, Jack J. Woehr, Stefan Woerner, Ryan Woo, Christopher J. Wood, Ryan Wood, Steve Wood, James Wootton, Matt Wright, Lucy Xing, Jintao YU, Bo Yang, Unchun Yang, Jimmy Yao, Daniyar Yeralin, Ryota Yonekura, David Yonge-Mallo, Ryuhei Yoshida, Richard Young, Jessie Yu, Lebin Yu, Christopher Zachow, Laura Zdanski, Helena Zhang, Iulia Zidaru, Christa Zoufal, aeddins ibm, alexzhang13, b63, bartek bartlomiej, bcamorrison, brandhsn, charmerDark, deeplokhande, dekel.meirom, dime10, dlasecki, ehchen, fanizzamarco, fs1132429, gadial, galeinston, georgezhou20, georgios ts, gruu, hhorii, hykavitha, itoko, jeppevinkel, jessica angel7, jezerjojo14, jliu45, jscott2, klinvill, krutik2966, ma5x, michelle4654, msuwama, nico lgrs, ntgiwsvp, ordmoj, sagar pahwa, pritamsinha2304, ryancocuzzo, saktar unr, saswati qiskit, septembrr, sethmerkel, sg495, shaashwat, smturro2, sternparky, strickroman, tigerjack, tsura crisaldo, upsideon, vadebayo49, welien, willhbang, wmurphy collabstar, yang.luh, and Mantas Čepulkovskis. Qiskit: An open-source framework for quantum computing, 2021.

[32] Thomas Pellizzari, Simon A Gardiner, Juan Ignacio Cirac, and Peter Zoller. Decoherence, continuous observation, and quantum computing: A cavity qed model. *Physical Review Letters*, 75(21):3788, 1995.

[33] Philip Wadler. A taste of linear logic. In *International Symposium on Mathematical Foundations of Computer Science*, pages 185–210. Springer, 1993.

[34] Bruno Barras, Samuel Boutin, Cristina Cornes, Judicaël Courant, Jean-Christophe Filliatre, Eduardo Gimenez, Hugo Herbelin, Gerard Huet, Cesar Munoz, Chetan Murthy, et al. *The Coq proof assistant reference manual: Version 6.1*. PhD thesis, Inria, 1997.

[35] Neil J Ross. Algebraic and logical methods in quantum computation. *arXiv preprint arXiv:1510.02198*, 2015.

[36] Andrew M Childs, Dmitri Maslov, Yunseong Nam, Neil J Ross, and Yuan Su. Toward the first quantum simulation with quantum speedup. *Proceedings of the National Academy of Sciences*, 115(38):9456–9461, 2018.

[37] Benjamin C Pierce, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hriţcu, Vilhelm Sjöberg, and Brent Yorgey. Software foundations. *Webpage: http://www. cis. upenn. edu/bcpierce/sf/current/index. html*, 2010.

[38] Scott Aaronson and Daniel Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70(5):052328, 2004.

[39] Sergey Bravyi, Graeme Smith, and John A Smolin. Trading classical and quantum computational resources. *Physical Review X*, 6(2):021043, 2016.

[40] Xin-Chuan Wu, Sheng Di, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic T Chong. Amplitude-aware lossy compression for quantum circuit simulation. *arXiv preprint arXiv:1811.05140*, 2018.

[41] Guifré Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical review letters*, 91(14):147902, 2003.

[42] Igor L Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.

[43] Sergio Boixo, Sergei V Isakov, Vadim N Smelyanskiy, and Hartmut Neven. Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384*, 2017.

[44] Amar Hadzihasanovic. The algebra of entanglement and the geometry of composition. *arXiv preprint arXiv:1709.08086*, 2017.

[45] Miriam Backens. The zx-calculus is complete for stabilizer quantum mechanics. *New Journal of Physics*, 16(9):093021, 2014.

[46] Alexei Yu Kitaev, Alexander Shen, Mikhail N Vyalyi, and Mikhail N Vyalyi. *Classical and quantum computation*. Number 47. American Mathematical Soc., 2002.

[47] Gushu Li, Li Zhou, Nengkun Yu, Yufei Ding, Mingsheng Ying, and Yuan Xie. Projection-based runtime assertions for testing and debugging quantum programs. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–29, 2020.

[48] Peng Fu, Kohei Kishida, and Peter Selinger. Linear dependent type theory for quantum programming languages. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 440–453, 2020.

[49] Gushu Li, Anbang Wu, Yunong Shi, Ali Javadi-Abhari, Yufei Ding, and Yuan Xie. Paulihedral: a generalized block-wise compiler optimization framework for quantum simulation kernels. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 554–569, 2022.

[50] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A Acar, et al. Quartz: Superoptimization of quantum circuits (extended version). *arXiv preprint arXiv:2204.09033*, 2022.

[51] Mohamed-Walid Benabderrahmane, Louis-Noël Pouchet, Albert Cohen, and Cédric Bastoul. The polyhedral model is more widely applicable than you think. In *International Conference on Compiler Construction*, pages 283–303. Springer, 2010.

[52] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 47–62, 2019.